

Informe de Práctica: Cliente-Servidor con Sockets en Java

- **Alumno:** Agustin Diaz
- **Fecha de entrega:** 15 de abril
- **Materia:** Programación avanzada

Introducción

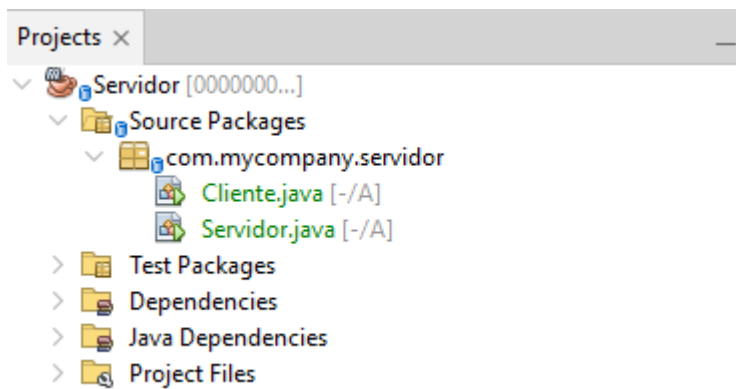
En esta práctica se implementa un sistema de comunicación Cliente-Servidor utilizando **Sockets en Java**. El objetivo es simular un chat donde el cliente envía mensajes al servidor, y además puede solicitar la resolución de expresiones matemáticas. El servidor responde y registra todo lo recibido en su consola.

Desarrollo

Estructura del proyecto

El proyecto se creó en **NetBeans con Maven**, con dos clases principales: Servidor y Cliente.

Estructura del Proyecto:



Clase Servidor:

```
1 public class Servidor {
2     public static void main(String[] args) {
3         int puerto = 5000;
4         try (ServerSocket serverSocket = new ServerSocket(puerto)) {
5             System.out.println("Servidor iniciado en puerto " + puerto);
6
7             Socket socket = serverSocket.accept();
8             System.out.println("Cliente conectado: " + socket.getInetAddress());
9
10            BufferedReader entrada = new BufferedReader(new InputStreamReader(socket.getInputStream()));
11            PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);
12
13            String mensaje;
14            while ((mensaje = entrada.readLine()) != null) {
15                System.out.println("Cliente dice: " + mensaje);
16
17                if (mensaje.equalsIgnoreCase("salir")) {
18                    salida.println("Conexión finalizada.");
19                    break;
20                } else if (mensaje.startsWith("RESOLVE")) {
21                    try {
22                        String expresion = mensaje.replace("RESOLVE", "").trim();
23                        double resultado = evaluarExpresion(expresion);
24                        salida.println("Resultado: " + resultado);
25                    } catch (Exception e) {
26                        salida.println("Error al resolver la expresión.");
27                    }
28                } else {
29                    salida.println("Servidor recibió: " + mensaje);
30                }
31            }
32
33            socket.close();
34            System.out.println("Servidor cerrado.");
35        } catch (IOException e) {
36            e.printStackTrace();
37        }
38    }
39 }
```

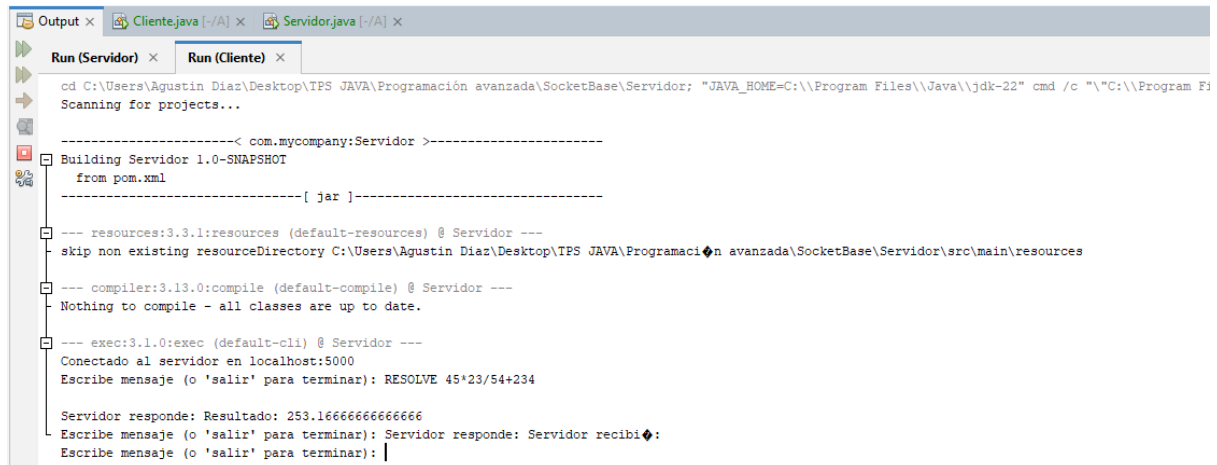
El servidor abre un socket en el puerto 5000, espera conexiones y responde a los mensajes del cliente. Si recibe un mensaje que comienza con RESOLVE, evalúa la expresión matemática usando la librería **exp4j**.

Clase Cliente

```
1 public class Cliente {
2     public static void main(String[] args) {
3         String host = "localhost";
4         int puerto = 5000;
5
6         try (Socket socket = new Socket(host, puerto);
7             BufferedReader entrada = new BufferedReader(new InputStreamReader(socket.getInputStream()));
8             PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);
9             Scanner scanner = new Scanner(System.in)) {
10
11             System.out.println("Conectado al servidor en " + host + ":" + puerto);
12
13             String mensaje;
14             while (true) {
15                 System.out.print("Escribe mensaje (o 'salir' para terminar): ");
16                 mensaje = scanner.nextLine();
17                 salida.println(mensaje);
18
19                 if (mensaje.equalsIgnoreCase("salir")) {
20                     System.out.println("Finalizando conexión...");
21                     break;
22                 }
23
24                 String respuesta = entrada.readLine();
25                 System.out.println("Servidor responde: " + respuesta);
26             }
27         } catch (IOException e) {
28             e.printStackTrace();
29         }
30     }
31 }
32 }
```

El cliente se conecta al servidor, permite al usuario escribir mensajes y muestra las respuestas recibidas. La comunicación se mantiene hasta que se envía el mensaje salir.

Pruebas y conclusiones



```
cd C:\Users\Agustin Diaz\Desktop\TPS JAVA\Programaci3n avanzada\SocketBase\Servidor; "JAVA_HOME=C:\Program Files\Java\jdk-22" cmd /c "%C:\Program F
Scanning for projects...

-----< com.mycompany.Servidor >-----
Building Servidor 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----
--- resources:3.3.1:resources (default-resources) @ Servidor ---
skip non existing resourceDirectory C:\Users\Agustin Diaz\Desktop\TPS JAVA\Programaci3n avanzada\SocketBase\Servidor\src\main\resources
--- compiler:3.13.0:compile (default-compile) @ Servidor ---
Nothing to compile - all classes are up to date.
--- exec:3.1.0:exec (default-cli) @ Servidor ---
Conectado al servidor en localhost:5000
Escribe mensaje (o 'salir' para terminar): RESOLVE 45*23/54+234

Servidor responde: Resultado: 253.16666666666666
Escribe mensaje (o 'salir' para terminar): Servidor responde: Servidor recibid
Escribe mensaje (o 'salir' para terminar):
```

Conclusiones

- Se logró implementar un sistema Cliente-Servidor en Java utilizando sockets.
- El servidor puede recibir mensajes y resolver expresiones matemáticas gracias a la librería exp4j.
- La práctica permitió comprender el flujo de comunicación entre procesos y cómo manejar entradas/salidas en red.